JAVASCRIPT — NOTES

Beginner → Intermediate → Advanced → Expert / Production

🧠 WHAT IS JAVASCRIPT?

JavaScript is a programming language used to add:

- behavior
- logic
- interaction
- dynamic content
- API communication
- browser functionality

to web applications.

```
HTML
 ↓
Structure

CSS
 ↓
Appearance

JavaScript
 ↓
Behavior + Logic
```

Example:

```
User clicks Generate
        ↓
JavaScript detects click
        ↓
Reads input
        ↓
Validates input
        ↓
Calls backend API
        ↓
Receives response
        ↓
Updates UI
```

LEVEL 1 — 🟢 FUNDAMENTALS

1. Running JavaScript

Inside HTML:

```
<script>
    console.log("Hello");
</script>
```

External file:

```
<script src="script.js"></script>
```

`script.js`:

```
console.log("Hello");
```

Professional applications normally separate JavaScript into modules/files.

---

2. Comments

Single line:

```
// This is a comment
```

Multiple lines:

```
/*
   Multiple-line
   comment
*/
```

---

3. console.log()

Used for output/debugging.

```
console.log("Hello");
console.log(10);
console.log(true);
```

---

4. Variables

```
let age = 20;
const name = "Alex";
```

let

Can be reassigned:

```
let score = 10;
score = 20;
```

const

Cannot be reassigned:

```
const name = "Alex";
```

var

Older declaration style.

```
var age = 20;
```

Modern professional JavaScript generally prefers:

```
const → default choice
let   → when reassignment is required
var   → generally avoid in modern code
```

---

5. Data Types

Primitive types

```
String
Number
Boolean
Undefined
Null
BigInt
Symbol
```

Examples:

```
const name = "Alex";
const age = 20;
const price = 99.5;
const active = true;
const value = undefined;
const data = null;
```

Object types

```
const user = {
    name: "Alex",
    age: 20
};
```

Arrays are also objects:

```
const numbers = [1, 2, 3];
```

---

6. typeof

Check the type:

```
typeof "Hello"
"string"
typeof 10
"number"
typeof true
"boolean"
```

Important historical JavaScript behavior:

```
typeof null
```

returns:

```
"object"
```

This is a well-known JavaScript quirk.

---

7. Operators

Arithmetic

```
+
-
*
/
%
**
```

Example:

```
10 + 3
10 - 3
10 * 3
10 / 3
10 % 3
10 ** 3
```

Comparison

```
>
<
```

```
>=
<=
===
!==
```

Prefer:

```
===
!==
```

instead of loose equality:

```
==
!=
```

---

8. Logical Operators

```
&&   // AND
||   // OR
!    // NOT
```

Example:

```
age >= 18 && isActive
```

Both conditions must be true.

---

9. Truthy and Falsy

Falsy values include:

```
false
0
""
null
undefined
NaN
```

Example:

```
const name = "";

if (name) {
    console.log("Exists");
} else {
    console.log("Missing");
}
```

Output:

```
Missing
```

This is extremely common in frontend code.

---

10. Type Conversion

Explicit conversion:

```
Number("100")
String(100)
Boolean(1)
```

Example:

```
const age = "20";

const numberAge = Number(age);
```

Invalid number:

```
Number("hello")
```

returns:

```
NaN
```

---

11. Template Literals

```
const name = "Alex";
const age = 20;

console.log(`My name is ${name} and I am ${age}.`);
```

Output:

```
My name is Alex and I am 20.
```

Similar to Python f-strings.

---

12. if / else

```
if (age >= 18) {
    console.log("Adult");
} else {
    console.log("Minor");
```

```
}
```

Multiple conditions:

```
if (score >= 90) {
    console.log("A");
} else if (score >= 80) {
    console.log("B");
} else {
    console.log("C");
}
```

---

13. Ternary Operator

```
const result = age >= 18 ? "Adult" : "Minor";
```

Pattern:

```
condition ? trueValue : falseValue
```

Useful for short UI conditions.

---

14. switch

```
switch (role) {
    case "admin":
        console.log("Admin");
        break;

    case "user":
        console.log("User");
        break;

    default:
        console.log("Unknown");
}
```

Useful when comparing one value against multiple known cases.

---

15. Loops

for

```
for (let i = 0; i < 5; i++) {
    console.log(i);
}
```

while

```
let i = 0;

while (i < 5) {
    console.log(i);
    i++;
}
```

for...of

Best suited for iterating values in iterable collections:

```
for (const item of items) {
    console.log(item);
}
```

for...in

Iterates property keys:

```
for (const key in user) {
    console.log(key);
}
```

For arrays, prefer `for...of` or array methods instead of `for...in`.

LEVEL 2 — 🟢 FUNCTIONS

16. Functions

A function is reusable logic.

```
function greet(name) {
    return `Hello ${name}`;
}
```

Call:

```
greet("Alex");
```

Result:

```
Hello Alex
```

---

17. Parameters and Arguments

```
function add(a, b) {
    return a + b;
}

add(10, 20);
```

Here:

```
a, b → parameters
10,20 → arguments
```

---

18. Default Parameters

```
function greet(name = "Guest") {
    return `Hello ${name}`;
}
greet();
```

Result:

```
Hello Guest
```

---

19. Arrow Functions

```
const add = (a, b) => {
    return a + b;
};
```

Short form:

```
const add = (a, b) => a + b;
```

Arrow functions are heavily used in modern JavaScript and React.

---

20. Rest Parameters

Collect multiple arguments:

```
function sum(...numbers) {
    return numbers.reduce((total, n) => total + n, 0);
}
sum(1, 2, 3, 4);
```

Result:

```
10
```

---

21. Scope

Scope determines where a variable can be accessed.

```
let name = "Alex";

if (true) {
    let age = 20;
}
```

`name` is available outside the block.

`age` is not.

Important concepts:

```
Global scope
Function scope
Block scope
Module scope
```

`let` and `const` are block scoped.

LEVEL 3 — 🟢 ARRAYS

22. Array

An array stores an ordered collection.

```
const fruits = ["Apple", "Banana", "Mango"];
```

Indexes:

```
Apple   → 0
Banana  → 1
Mango   → 2
```

JavaScript arrays are zero-indexed.

---

23. Access and Update

```
fruits[0]
```

→ `"Apple"`

Update:

```
fruits[1] = "Orange";
```

---

24. length

```
fruits.length
```

Last element:

```
fruits[fruits.length - 1]
```

---

25. Add / Remove

Method	Purpose
`push()`	Add at end
`pop()`	Remove end
`unshift()`	Add beginning
`shift()`	Remove beginning

Example:

```
fruits.push("Orange");
```

---

26. slice()

Copies/extracts part of an array.

```
const result = fruits.slice(1, 3);
```

End index is excluded.

```
slice(start, end)
```

Does not mutate the original array.

---

27. splice()

Adds/removes elements and mutates the original array.

```
fruits.splice(1, 1);
```

Meaning:

```
start at index 1
remove 1 item
```

Difference:

```
slice  → non-mutating
splice → mutating
```

---

28. includes()

```
fruits.includes("Apple");
```

Returns:

```
true / false
```

---

29. indexOf()

```
fruits.indexOf("Mango");
```

Returns index or:

```
-1
```

if not found.

---

30. forEach()

Runs a function for every element.

```
fruits.forEach(fruit => {
    console.log(fruit);
});
```

Use when you want to **perform an action** for each item.

---

31. map()

Creates a new transformed array.

```
const numbers = [1, 2, 3];

const doubled = numbers.map(n => n * 2);
```

Result:

```
[2, 4, 6]
```

Mental model:

```
[1, 2, 3]
   ↓ map
[2, 4, 6]
```

---

32. filter()

Creates a new array containing matching elements.

```
const ages = [12, 18, 25, 15];

const adults = ages.filter(age => age >= 18);
```

Result:

```
[18, 25]
```

---

33. find()

Returns the first matching element.

```
const user = users.find(user => user.id === 2);
```

If none exists:

```
undefined
```

---

34. some()

Checks whether at least one element matches.

```
const result = ages.some(age => age >= 18);
```

Returns:

```
true / false
```

---

35. every()

Checks whether every element matches.

```
const result = ages.every(age => age >= 18);
```

Returns:

```
true / false
```

---

36. reduce()

Combines many values into one result.

```
const numbers = [10, 20, 30];

const total = numbers.reduce(
    (sum, number) => sum + number,
    0
);
```

Result:

```
60
```

Real application:

```
const cart = [
    { name: "Keyboard", price: 50 },
    { name: "Mouse", price: 30 }
];

const total = cart.reduce(
    (sum, item) => sum + item.price,
    0
);
```

Result:

```
80
```

---

37. sort()

For numbers:

```
numbers.sort((a, b) => a - b);
```

Ascending:

```
[10, 20, 30]
```

Descending:

```
numbers.sort((a, b) => b - a);
```

Important:

`sort()` mutates the array.

Non-mutating modern alternative:

```
const sorted = numbers.toSorted((a, b) => a - b);
```

---

38. Array Spread

```
const a = [1, 2];
const b = [3, 4];

const combined = [...a, ...b];
```

Result:

```
[1, 2, 3, 4]
```

Copy:

```
const copy = [...a];
```

This creates a shallow copy.

---

39. Array Destructuring

```
const numbers = [10, 20, 30];

const [first, second] = numbers;
```

Now:

```
first  → 10
second → 20
```

🧠 ARRAY METHODS — QUICK TABLE

Method	Purpose	Mutates?

push	Add end	Yes
pop	Remove end	Yes
shift	Remove beginning	Yes
unshift	Add beginning	Yes
slice	Extract/copy	No
splice	Insert/remove	Yes
map	Transform	No
filter	Select	No
find	Find first	No
some	At least one	No
every	All	No
reduce	Combine	No
sort	Sort	Yes
toSorted	Sort copy	No

The four especially important for React:

```
map
filter
find
reduce
```

LEVEL 4 — 🟢 OBJECTS

40. Object

Objects represent entities using key-value pairs.

```
const user = {
    name: "Alex",
    age: 20,
    active: true
};
```

Think:

```
user
 ├── name
 ├── age
 └── active
```

---

41. Access Properties

Dot notation:

```
user.name
```

Bracket notation:

```
user["name"]
```

Bracket notation is useful when the property name is dynamic.

```
const key = "name";

user[key];
```

---

42. Add / Modify / Delete

Add:

```
user.email = "alex@example.com";
```

Modify:

```
user.age = 21;
```

Delete:

```
delete user.active;
```

Use `delete` carefully in performance-sensitive code; in application state, creating a new object without the property is often preferable.

---

43. Object Methods

Functions can be properties.

```
const user = {
    name: "Alex",

    greet() {
        console.log("Hello");
    }
};

user.greet();
```

---

44. Nested Objects

```
const user = {
    name: "Alex",
    address: {
```

```
        city: "Delhi",
        country: "India"
    }
};
```

Access:

```
user.address.city
```

---

45. Object Destructuring

```
const user = {
    name: "Alex",
    age: 20
};

const { name, age } = user;
```

Now:

```
name → Alex
age  → 20
```

Very common in React.

---

46. Object Spread

```
const user = {
    name: "Alex",
    age: 20
};

const updatedUser = {
    ...user,
    age: 21
};
```

Result:

```
{
    name: "Alex",
    age: 21
}
```

This is a major pattern for immutable updates.

---

47. Computed Properties

```
const key = "name";

const user = {
    [key]: "Alex"
};
```

Result:

```
{
    name: "Alex"
}
```

Useful when property names are dynamic.

---

48. Object.keys()

```
Object.keys(user);
```

Returns an array of property names.

---

49. Object.values()

```
Object.values(user);
```

Returns property values.

---

50. Object.entries()

```
Object.entries(user);
```

Returns:

```
[
    ["name", "Alex"],
    ["age", 20]
]
```

Useful for iterating through objects.

---

51. Optional Chaining

```
user.address?.city
```

If `address` doesn't exist, JavaScript returns:

```
undefined
```

instead of throwing an error.

Useful with API responses.

---

52. Nullish Coalescing

```
const name = user.name ?? "Guest";
```

If the left side is `null` or `undefined`, use `"Guest"`.

Important difference:

```
?? → null / undefined
|| → any falsy value
```

---

53. Object Reference

```
const user1 = {
    name: "Alex"
};

const user2 = user1;

user2.name = "John";
```

Now:

```
user1.name → John
user2.name → John
```

Both references point to the same object.

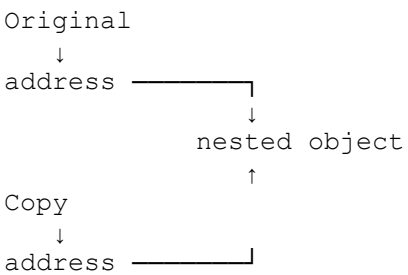
---

54. Shallow Copy

```
const copy = { ...user };
```

Creates a new outer object.

But nested objects can still be shared.



This concept is very important when working with application state.

LEVEL 5 — 🟡 MODERN JAVASCRIPT

55. Destructuring

Arrays:

```
const [a, b] = [10, 20];
```

Objects:

```
const { name, age } = user;
```

Rename:

```
const { name: userName } = user;
```

---

56. Spread vs Rest

Same `...` syntax, different jobs.

Spread

Expands values:

```
const numbers = [1, 2, 3];

const copy = [...numbers];
```

Rest

Collects values:

```
function sum(...numbers) {}
```

Think:

```
spread → unpack
rest   → collect
```

---

57. Modules

Export:

```
export function add(a, b) {
    return a + b;
}
```

Import:

```
import { add } from "./utils.js";
```

Default export:

```
export default function App() {}
```

Import:

```
import App from "./App.js";
```

Modules allow large applications to be split into manageable pieces.

---

58. Strict Mode

```
"use strict";
```

Makes certain problematic JavaScript behaviors produce errors instead of silently continuing.

ES modules are automatically strict mode.

LEVEL 6 — 🟡 DOM

59. DOM

DOM = Document Object Model.

The browser converts HTML into a tree-like object structure.

```
HTML
└── body
    ├── h1
    ├── p
    └── button
```

JavaScript can read and modify this structure.

---

60. Selecting Elements

```
document.getElementById("title");
document.querySelector(".card");
document.querySelectorAll(".card");
```

`querySelector()` returns the first matching element.

`querySelectorAll()` returns a collection of matching elements.

---

61. Changing Content

```
element.textContent = "Hello";
```

Use `textContent` when you want text.

Be careful with:

```
element.innerHTML = userInput;
```

because inserting untrusted HTML can create security problems such as XSS.

---

62. Classes

```
element.classList.add("active");
```

Remove:

```
element.classList.remove("active");
```

Toggle:

```
element.classList.toggle("active");
```

This is usually cleaner than directly changing many CSS properties from JavaScript.

---

63. Creating Elements

```
const button = document.createElement("button");

button.textContent = "Click";

document.body.append(button);
```

JavaScript can dynamically create UI elements.

LEVEL 7 — 🟡 EVENTS

64. Events

An event means something happened.

Examples:

```
click
submit
input
change
keydown
keyup
focus
blur
scroll
```

---

65. addEventListener()

```
button.addEventListener("click", () => {
    console.log("Clicked");
});
```

Mental model:

```
Event happens
     ↓
Browser detects it
     ↓
Listener runs
```

```
     ↓
Your function executes
```

---

66. Event Object

```
button.addEventListener("click", (event) => {
    console.log(event);
});
```

The event object contains information about what happened.

For keyboard events:

```
document.addEventListener("keydown", event => {
    console.log(event.key);
});
```

---

67. preventDefault()

Forms normally perform browser-default behavior.

```
form.addEventListener("submit", event => {
    event.preventDefault();
});
```

Now you can handle submission using JavaScript.

---

68. Event Bubbling

Suppose:

```
button
  ↓
div
  ↓
body
```

A click on the button can bubble upward:

```
button
 ↓
div
 ↓
body
```

This is event bubbling.

---

69. Event Delegation

Instead of attaching listeners to 100 buttons, attach one listener to a parent.

```
container.addEventListener("click", event => {
    if (event.target.matches(".delete-button")) {
        // delete
    }
});
```

Useful for dynamic lists and large interfaces.

LEVEL 8 — 🟡 ASYNCHRONOUS JAVASCRIPT

JavaScript applications constantly wait for:

```
API responses
files
timers
network requests
database operations
AI responses
```

---

70. Synchronous vs Asynchronous

Synchronous:

```
Task A
 ↓
Task B
 ↓
Task C
```

Asynchronous:

```
Start A
 ↓
while waiting
 ↓
other work can continue
 ↓
A completes
 ↓
callback/continuation runs
```

---

71. setTimeout()

```
setTimeout(() => {
    console.log("Hello");
}, 1000);
```

Runs after the timer becomes ready.

It does not mean JavaScript freezes for exactly one second.

---

72. Promise

A Promise represents the eventual result of an asynchronous operation.

States:

```
pending
   ↓
fulfilled

OR

rejected
```

Example:

```
const promise = fetch("/api/data");
```

---

73. then / catch

```
fetch("/api/data")
    .then(response => response.json())
    .then(data => console.log(data))
    .catch(error => console.error(error));
```

---

74. async / await

Modern style:

```
async function getData() {
    const response = await fetch("/api/data");

    const data = await response.json();

    return data;
}
```

`await` pauses the async function's continuation until the Promise settles; it does not block the entire JavaScript runtime.

---

75. Error Handling with async/await

```
async function getData() {
    try {
        const response = await fetch("/api/data");

        if (!response.ok) {
            throw new Error("Request failed");
        }

        const data = await response.json();

        return data;

    } catch (error) {
        console.error(error);
    }
}
```

Professional applications must handle:

```
network errors
server errors
invalid responses
authentication failures
timeouts
unexpected data
```

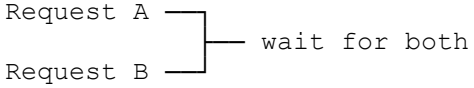
---

76. Promise.all()

Run independent asynchronous operations together.

```
const [users, products] = await Promise.all([
    fetch("/api/users"),
    fetch("/api/products")
]);
```

Conceptually:



Useful when requests don't depend on each other.

---

77. Promise.allSettled()

Useful when you want every operation's result even if some fail.

```
const results = await Promise.allSettled([
    task1(),
    task2(),
    task3()
]);
```

LEVEL 9 — 🟡 FETCH + APIs + JSON

78. fetch()

GET:

```
const response = await fetch("/api/users");
```

Then:

```
const data = await response.json();
```

---

79. POST

```
const response = await fetch("/api/emails", {
    method: "POST",
    headers: {
        "Content-Type": "application/json"
    },
    body: JSON.stringify({
        prompt: "Write a professional email"
    })
});
```

Typical flow:

```
Frontend
   ↓
fetch()
   ↓
HTTP request
   ↓
FastAPI
   ↓
Business logic
   ↓
Response
   ↓
JavaScript
```

```
   ↓
UI
```

---

80. HTTP Status

Common status categories:

```
2xx → success
3xx → redirection
4xx → client/request problem
5xx → server problem
```

Always check:

```
if (!response.ok) {
    throw new Error("Request failed");
}
```

---

81. JSON

Convert JavaScript data to JSON:

```
JSON.stringify(data);
```

Convert JSON to JavaScript value:

```
JSON.parse(data);
```

Fetch response:

```
const data = await response.json();
```

JSON is a common format for frontend-backend communication.

LEVEL 10 — 🔴 ADVANCED JAVASCRIPT

82. Closures

A closure occurs when a function retains access to variables from its surrounding lexical scope.

```
function counter() {
    let count = 0;

    return function () {
```

```
        count++;
        return count;
    };
}

const increment = counter();

increment(); // 1
increment(); // 2
increment(); // 3
```

The returned function still has access to `count`.

Important for:

```
callbacks
state
encapsulation
React
functional programming
```

---

83. Callback

A function passed to another function.

```
function process(name, callback) {
    callback(name);
}

process("Alex", name => {
    console.log(name);
});
```

---

84. Higher-Order Functions

A function that:

- receives another function
- returns another function
- or both

Examples:

```
map()
filter()
find()
reduce()
```

These are higher-order functions because they receive functions.

---

85. `this`

`this` refers to a value determined by how a function is called.

Example:

```
const user = {
    name: "Alex",

    greet() {
        console.log(this.name);
    }
};

user.greet();
```

Output:

```
Alex
```

Arrow functions do not create their own `this`; they capture it lexically from the surrounding scope.

This distinction is important.

---

86. Prototypes

JavaScript uses prototype-based object inheritance.

Conceptually:

```
object
   ↓
prototype
   ↓
prototype's prototype
   ↓
...
   ↓
null
```

When JavaScript cannot find a property directly on an object, it can look up the prototype chain.

---

87. Classes

JavaScript provides class syntax:

```
class User {
    constructor(name) {
        this.name = name;
    }

    greet() {
        return `Hello ${this.name}`;
    }
}
```

Create:

```
const user = new User("Alex");
```

Classes are built on JavaScript's prototype system.

---

88. Inheritance

```
class Admin extends User {
    deleteUser() {
        console.log("Deleting user");
    }
}
```

`Admin` inherits from `User`.

Modern JavaScript often favors composition over deep inheritance hierarchies.

---

89. Map

JavaScript also has a built-in `Map` data structure.

```
const users = new Map();

users.set(1, "Alex");
users.set(2, "John");
```

Get:

```
users.get(1);
```

Result:

```
Alex
```

Useful when you need key-value storage with capabilities that differ from ordinary objects.

---

90. Set

Stores unique values.

```
const numbers = new Set([1, 2, 2, 3]);
```

Result conceptually:

```
{1, 2, 3}
```

Useful for removing duplicates:

```
const unique = [...new Set(numbers)];
```

LEVEL 11 — 🔴 JAVASCRIPT ENGINE + EVENT LOOP

91. Call Stack

JavaScript executes function calls using a call stack.

Example:

```
function a() {
    b();
}

function b() {
    console.log("Hello");
}

a();
```

Conceptually:

```
a()
 ↓
b()
 ↓
console.log()
```

Functions are added and removed from the stack as they execute.

---

92. Event Loop

Simplified browser model:

```
JavaScript
    ↓
Call Stack
    ↓
Web APIs / runtime
    ↓
Queues
    ↓
Event Loop
    ↓
Call Stack
```

Example:

```
console.log("A");

setTimeout(() => {
    console.log("B");
}, 0);

console.log("C");
```

Output:

```
A
C
B
```

The timer callback waits until the current synchronous work has completed and the runtime schedules it.

---

93. Microtasks vs Tasks

Promise reactions are generally placed in the microtask queue.

Timers such as `setTimeout` use task scheduling.

Conceptually:

```
Synchronous code
       ↓
Microtasks
       ↓
Tasks
```

This is why:

```
console.log("A");

Promise.resolve().then(() => {
    console.log("B");
});

setTimeout(() => {
    console.log("C");
}, 0);

console.log("D");
```

produces:

A
D
B
C

Understanding this is useful for advanced debugging and interviews.

LEVEL 12 — 🔴 IMMUTABILITY + APPLICATION STATE

This becomes extremely important before React.

Mutation:

```
user.name = "John";
```

New object:

```
const updatedUser = {
    ...user,
    name: "John"
};
```

Array:

```
const updatedItems = items.filter(item => item.id !== id);
```

Add:

```
const updatedItems = [...items, newItem];
```

Update:

```
const updatedItems = items.map(item =>
    item.id === id
        ? { ...item, completed: true }
        : item
```

```
);
```

This style is heavily used in React state management.

LEVEL 13 — 🔴 PROFESSIONAL JAVASCRIPT

94. Separation of Concerns

Don't put everything in one function.

Bad structure:

```
button click
 ↓
validation
 ↓
API request
 ↓
data transformation
 ↓
UI rendering
 ↓
all inside one giant function
```

Better:

```
UI
 ↓
event handler
 ↓
validation function
 ↓
API module
 ↓
data processing
 ↓
UI update
```

---

95. Reusable Modules

Example structure:

```
src/
├── api/
│   └── emailApi.js
├── utils/
│   └── validation.js
├── components/
├── services/
└── main.js
```

The exact structure changes with the framework, but the principle remains:

Keep responsibilities separated.

---

96. Debouncing

Useful for search inputs.

Without debounce:

```
User types:
A
 ↓ API

Al
 ↓ API

Ale
 ↓ API

Alex
 ↓ API
```

Potentially four requests.

With debounce:

```
User types Alex
      ↓
wait briefly
      ↓
one API request
```

Useful for:

```
search
autocomplete
live validation
resize handling
```

---

97. Throttling

Limits how often a function can execute.

Useful for:

```
scroll
mousemove
resize
```

```
continuous events
```

Difference:

```
debounce
→ wait until activity stops

throttle
→ allow execution at controlled intervals
```

---

98. Error Boundaries vs JavaScript Errors

Ordinary JavaScript error handling:

```
try {
    // code
} catch (error) {
    // handle
}
```

Frameworks such as React also provide higher-level mechanisms for handling rendering failures.

Don't confuse application-level error handling with all possible UI failure recovery.

---

99. Security

Important frontend security concepts:

```
XSS
CSRF
CORS
input validation
unsafe HTML
authentication
authorization
token handling
secret management
```

Never expose private secrets

Bad:

```
const OPENAI_API_KEY = "secret";
```

Frontend code is visible to the user.

Better:

```
Browser
  ↓
Your backend
  ↓
Secret API key
  ↓
AI provider
```

This principle is especially important for your AI applications.

---

100. Performance

Important concepts:

```
avoid unnecessary work
efficient DOM updates
debouncing
throttling
lazy loading
code splitting
memory management
large data handling
caching
```

Don't optimize blindly.

First:

```
measure
 ↓
identify bottleneck
 ↓
optimize
 ↓
measure again
```

---

101. Accessibility

JavaScript must not destroy accessibility.

Consider:

```
keyboard navigation
focus management
semantic HTML
screen readers
ARIA when appropriate
reduced motion
form errors
loading states
```

Prefer semantic HTML and native browser behavior before adding custom JavaScript.

---

102. Debugging

Browser **DevTools** are essential.

Important tools:

```
Console
Elements
Network
Sources
Application
Performance
Memory
```

Typical debugging process:

```
Bug
 ↓
Reproduce
 ↓
Inspect
 ↓
Find actual cause
 ↓
Fix
 ↓
Test again
```

Avoid guessing.

---

103. Testing Mindset

Even before learning the Testing phase of your roadmap, write code that is easy to test.

Prefer:

```
function calculateTotal(items) {
    return items.reduce(
        (sum, item) => sum + item.price,
        0
    );
}
```

over mixing calculation, DOM operations, API requests, and unrelated logic in one giant function.

Pure functions are especially easy to test.

🧠 MOST IMPORTANT JAVASCRIPT CONCEPT MAP

```
VARIABLES
    ↓
TYPES
    ↓
CONDITIONS
    ↓
LOOPS
    ↓
FUNCTIONS
    ↓
ARRAYS + OBJECTS
    ↓
DOM
    ↓
EVENTS
    ↓
ASYNC JAVASCRIPT
    ↓
FETCH + APIs
    ↓
MODULES
    ↓
CLOSURES
    ↓
EVENT LOOP
    ↓
IMMUTABILITY
    ↓
PROFESSIONAL ARCHITECTURE
```

🧩 MOST IMPORTANT DIFFERENCES

Concept A	Concept B	Difference
`let`	`const`	reassignment
`==`	`===`	loose vs strict equality
`slice()`	`splice()`	non-mutating vs mutating
`map()`	`forEach()`	transform vs perform action
`filter()`	`find()`	many vs first matching item
`some()`	`every()`	at least one vs all
`spread`	`rest`	expand vs collect
`??`	`` ` ``	
`async/await`	`.then()`	two styles for working with Promises
debounce	throttle	after activity stops vs controlled rate
object	`Map`	different key-value data-structure behavior
shallow copy	deep copy	outer level vs nested data independence

---

🎤 EXPECTED INTERVIEW QUESTIONS

Fundamentals

1. What is JavaScript?
2. Difference between `let`, `const`, and `var`?
3. What are primitive data types?
4. What is type coercion?
5. Difference between `==` and `===`?
6. What are truthy and falsy values?
7. What is `NaN`?
8. What is scope?

Arrays / Objects

9. What is an array?
10. Why are JavaScript arrays zero-indexed?
11. Difference between `slice()` and `splice()`?
12. Difference between `map()` and `forEach()`?
13. Difference between `filter()` and `find()`?
14. What is `reduce()`?
15. What is destructuring?
16. What is the spread operator?
17. What is shallow copying?
18. How are objects passed/referenced?
19. Difference between `Object.keys()`, `values()`, and `entries()`?

Browser

20. What is the DOM?
21. What is event bubbling?
22. What is event delegation?
23. What does `preventDefault()` do?
24. Difference between `textContent` and `innerHTML`?

Async

25. What is asynchronous JavaScript?
26. What is a Promise?
27. What is `async/await`?
28. How does `fetch()` work?
29. What is `Promise.all()`?
30. What is the event loop?
31. What are microtasks?

Advanced

32. What is a closure?
33. What is a callback?
34. What is a higher-order function?
35. How does `this` work?
36. What are prototypes?
37. What are JavaScript modules?
38. What is immutability?
39. What is debouncing?
40. What is throttling?

Professional

41. How should frontend secrets be handled?
42. What is XSS?
43. Why is input validation important?
44. How would you structure a large JavaScript application?
45. How would you debug a slow frontend?
46. How would you handle API failures?
47. How would you reduce unnecessary API requests?
48. How would you make a dynamic UI accessible?
